GANGA INSTITUTE OF TECHNOLOGY & MANAGEMENT
JHAJJAR, HARYANA

A PROJECT REPORT ON

"SMART LIGHTING SYSTEM"

SUBMITTED OF THE REQUIREMENTS FOR THE TASK-1 ASSIGNMENT

MAINCRAFTS TECHNOLOGY INTERNSHIP :

INTRODUCTION TO EMBEDDED SYSTEMS & IOT DEVICE DESIGN

SUBMITTED TO :
MAINCRAFTS TECHNOLOGY

SUBMITTED BY :
NAME - ROHAN KUMAR
COLLEGE ROLL - 24ECE031
PROGRAM – INTRODUCTION TO EMBEDDED SYSTEMS & IOT DEVICE DESIGN

Smart Lighting System Using Embedded Technology

Introduction to Embedded Systems

What is an Embedded System?

An embedded system is a specialized computer system engineered to perform dedicated, specific functions within a larger mechanical or electrical ecosystem. Unlike general-purpose personal computers (PCs) that handle arbitrary software applications, an embedded system relies on an optimal combination of custom hardware and deeply integrated software (firmware) to execute a predefined set of tasks reliably, efficiently, and often in real-time.

Introduction

A Smart Lighting System is an automated embedded technology that acts as a bridge between environmental conditions and lighting loads. Its main objective is efficiency: eliminating manual human intervention by turning lights ON or OFF purely based on sensor feedback

Working Principle

The system continuously monitors ambient light levels using an LDR connected to an Analog Input pin on the Arduino.

1. When daylight is present, the LDR resistance is low, resulting in a high analog reading. The Arduino detects this and keeps the LED **OFF**.
2. When it gets dark, the LDR resistance spikes, causing the analog reading to drop below a predefined **threshold**. The Arduino detects this change and turns the LED **ON**.

Real-World Applications

Embedded systems form the backbone of modern automation, critical infrastructure, and consumer electronics:

- **Smart Home Technology:** Systems like automated climate control (thermostats), smart security cameras, connected appliances, and ambient-reactive lighting networks.
- **Automotive Engineering:** Engine Control Units (ECUs), Anti-lock Braking Systems (ABS), airbag deployment triggers, and autonomous driver-assistance systems (ADAS).
- **Healthcare & Medical Devices:** Life-critical applications including implantable pacemakers, automated insulin pumps, diagnostic MRI machines, and real-time vital sign monitors.
- **Industrial Automation & Robotics:** Programmable Logic Controllers (PLCs), robotic assembly arms, and automated guided vehicles (AGVs) operating on factory floors.

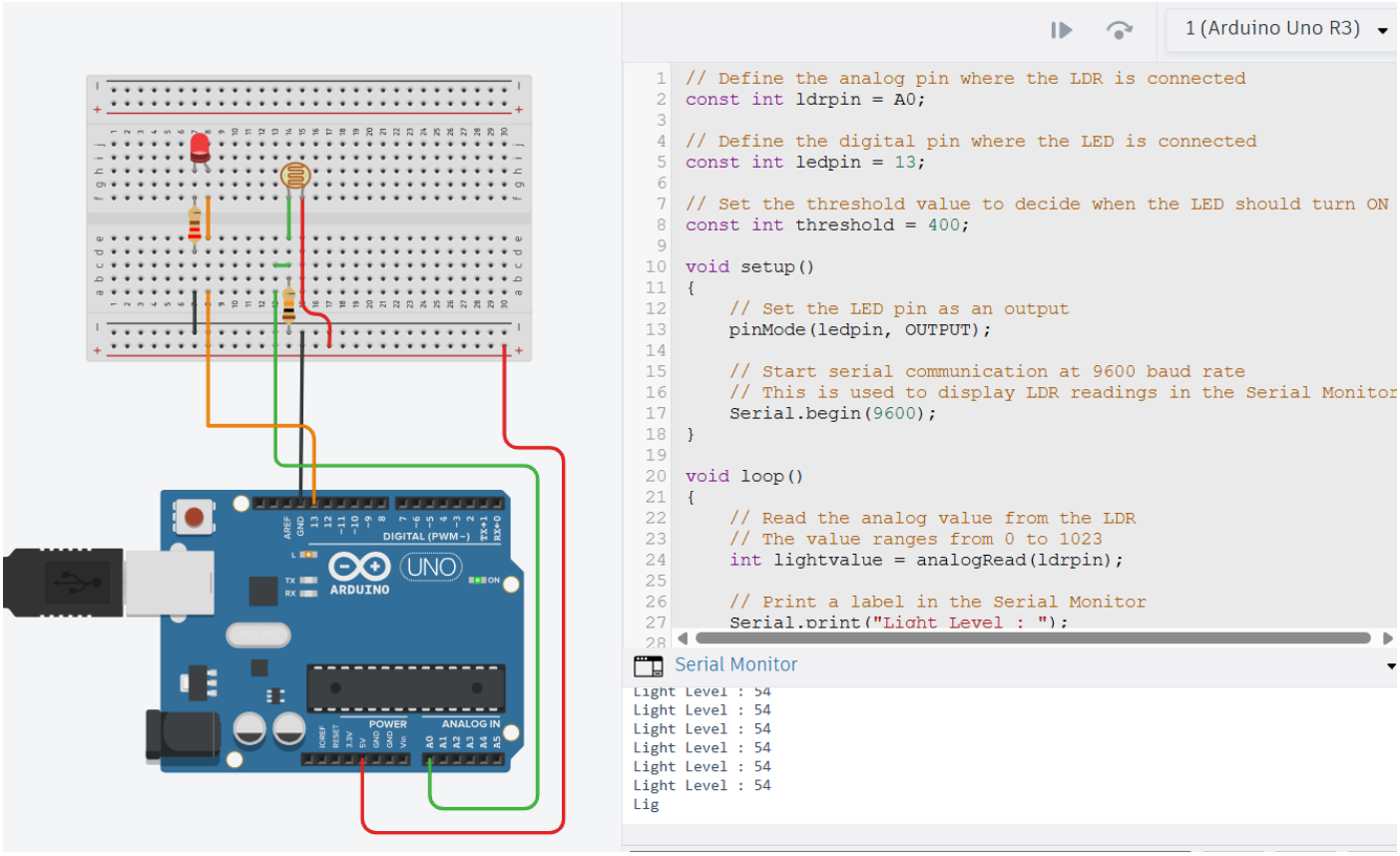
Microprocessors vs. Microcontrollers

Architectural Attribute	Microcontroller (MCU) (e.g., ATmega328P, ESP32)	Microprocessor (MPU) (e.g., Intel Core i9, ARM Cortex-A72)
Integration Layout	**System-on-Chip (SoC):** Integrates the CPU, RAM, ROM/Flash memory, and Input/Output (I/O) peripherals onto a single integrated circuit	**Discrete Architecture:** Contains only the central processing unit (CPU). RAM, storage (ROM), and I/O controllers must be connected externally.
Primary Design Goal	Optimized for specific, low-power control tasks and real-time physical interaction with hardware components.	Optimized for high-speed, general-purpose computation, complex calculations, and heavy multitasking.
Power Consumption	Extremely low power requirements (milliwatts to microwatts), making them ideal for battery-operated devices.	High power consumption (typically several watts to over 100W), requiring active cooling systems.
Cost & Footprint	Low cost per unit with minimal spatial requirements on a printed circuit board (PCB).	High unit cost and complex PCB design layouts due to extensive external routing busses.

Required Components

- Arduino
- Light dependent registor (LDR)
- Light Emitting Diode (LED)
- 2 registors (220 ohm for LED and 10K ohm for LDR)
- Breadboard
- wires
- Tinkercad – Software for simulation

Circuit Diagram



Programming

```
// Define the analog pin where the LDR is connected
const int ldrpin = A0;
// Define the digital pin where the LED is connected
const int ledpin = 13;
// Set the threshold value to decide when the LED should turn ON
const int threshold = 400;
void setup() {
   // Set the LED pin as an output
   pinMode(ledpin, OUTPUT);
   // Start serial communication at 9600 baud rate
   // This is used to display LDR readings in the Serial Monitor
   Serial.begin(9600); }
void loop() {
   // Read the analog value from the LDR
   // The value ranges from 0 to 1023
   int lightvalue = analogRead(ldrpin);
   // Print a label in the Serial Monitor
   Serial.print("Light Level : ");
   // Print the current LDR value
   Serial.println(lightvalue);
   // Check if the light intensity is below the threshold
   // (Lower value = Dark, depending on the circuit connection)
   if (lightvalue < threshold) {
      // Turn ON the LED
      digitalWrite(ledpin, HIGH); }
   else {
      // Turn OFF the LED
      digitalWrite(ledpin, LOW); }
 // Wait for 200 milliseconds before taking the next reading
 delay(200);
```